

Parametric Polymorphism

a.k.a. *Generic* Functions and Types

Kenjiro Taura

2024/05/09

Motivation

say want to write ...

- a function that *sorts arrays of various types* (e.g., ints, floats, strings, structs, ...)
- a function that *extracts elements from a list satisfying* $p(x)$
- *stacks, queues, trees, graphs, hashtables, etc.*
- many *graph algorithms (breadth-first search, depth-first search, connected components, partitioning, etc.)*
- ... *without duplicating code* for each element type

A trivial example (generic function)

write a generic function $f(a) = a[0]$ in your language (an element of an array) that works for any element type

Questions:

- do you have to specify the type of a ?
- if so, how can you say *a must be an array but whose element can be any type*
- if not, can it automatically apply to any array?
 - *does it type-check statically* (i.e., what if you pass something not an array)?

Type expressions

- things are conceptually straightforward
- but *spelling out types* needs a practice (for languages that require type annotations)
- master the syntax of *type expressions, parameterized types/ functions, and instantiation thereof*

Type expressions for functions

ex. a type of *functions taking an integer and returning a float*

Go	<code>func (int64) float64</code>	
Julia	<code>Function</code>	<code>(*)</code> , <code>(†)</code>
OCaml	<code>int -> float</code>	<code>(†)</code>
Rust	<code>fn (i64) -> f64</code>	

- `(*)` cannot specify input/output types
- `(†)` you normally don't write it

Type expressions for array-like data

ex. (one-dimensional) array (or likes) of 64-bit floats

Go	fixed-size (n -element) array slice	<code>[n]float64</code> <code>[]float64</code>
Julia		<code>Vector{Float64}</code>
OCaml		<code>float array</code>
Rust	fixed-size (n -element) array vector slice	<code>[f64; n]</code> <code>Vec<f64></code> <code>[f64]</code>

Defining parameterized types

ex. Node (or Tree) of any type

Go	<code>type Node[T any] struct { ... }</code>
Julia	<code>struct Node{T} ... end</code>
OCaml	<code>type 'a tree = ... class ['a] node ... = object ... end</code>
Rust	<code>enum Tree<T> { ... } struct Node<T> { ... }</code>

Defining parameterized types

and a version parameterized by *any subtype of S*

Go	<code>type Node[T S] struct { ... }</code>
Julia	<code>struct Node{T<:S} ... end</code>
OCaml	not available
Rust	<code>enum Tree<T : S> { ... }</code> <code>struct Node<T : S> { ... }</code>

Instantiating parameterized types

ex. Node of 64-bit integers

Go	Node[int64]	
Julia	Node{Int64}	
OCaml	int node	
Rust	Node<i64> or Node:: <i>i64></i>	(*)

- $(*) ::$ is necessary to disambiguate the symbol $<$
- $\approx ::$ is unnecessary where only type expressions are expected and necessary when ordinary expressions are expected

Defining parameterized functions

ex. a function `dfs`, which can work for node of *any type*

Go	<code>func dfs[T any](n Node[T]) { ... }</code>	
Julia	<code>function dfs(n : Node{T}) where T ... end</code>	
OCaml	<code>let dfs (n : 'a tree) = ... let dfs n = ...</code>	(*)
Rust	<code>fn dfs<T>(n : Tree<T>) { ... }</code>	

- (*) : normally not necessary

Defining parameterized functions

and a version that can work for *any subtype of S*

Go	<code>func dfs[T S](n Node[T]) { ... }</code>
Julia	<code>function dfs(n : Node{T}) where {T<:S} ... end</code>
OCaml	not available
Rust	<code>fn dfs<T : S>(n : Tree<T>) { ... }</code>

Instantiating parameterized functions

Go	<code>bfs[int64](...)</code>	
Julia	<code>function bfs(...)</code>	no specific syntax
OCaml	<code>bfs ...</code>	no specific syntax
Rust	<code>bfs::<i64>(...)</code>	

Type inference

- *type inference* generally refers to any algorithm that determines the static type of an expression without programmer's annotation
- many languages infer types of local variables
 - Go : type of a local variable initialized with `:=` is inferred by RHS
 - Rust : type of a local variable is inferred by RHS (or the unique assignment)
- but they assume types of function parameters are given

Type inference

- they infer type of a compound expression from the types of its sub-expressions

OCaml type inference (reconstruction)

- OCaml's type inference is remarkable in that it infers types of *incoming* parameters from function body
- intuitively, it works by collecting *constraints*
 - ▶ $x + y \rightarrow x : \text{int}, y : \text{int}$ (assumption: $+ : \text{int} \rightarrow \text{int}$)
 - ▶ $\text{match } l \text{ with } [] \rightarrow \dots \mid a::r \rightarrow \dots \rightarrow l : 'a \text{ list}$
 - ▶ $b.[i] \rightarrow b : 'a \text{ array}$
 - ▶ $b.[i] + 1 \rightarrow b : \text{int array}$
 - ▶ $s\#\text{area} < 10.0 \rightarrow s : < \text{area} : \text{float}; \dots >$

OCaml type inference (reconstruction)

- thanks to type inference, you can omit most type annotations in OCaml, but it's not that it does not do static type checking (unlike Python or Julia)
- it is indeed type safe
- to reason about what kind of programs are type-checked, you have to know possible types and their syntax (e.g., to understand error messages)
- it is possible, and indeed useful, to give type annotations for documentation and diagnosing type errors